

Databases 2 - exam - February 5, 2025 - Dur. 2h

S. Comai, P. Fraternali, D. Martinenghi

A. Concurrency control (11 points)

Consider a distributed system with three nodes (A, B, C) with the following wait-for conditions:

NODE A: $E_B \rightarrow t_4$; $t_4 \rightarrow t_7$; $t_7 \rightarrow t_5$; $t_5 \rightarrow E_C$

NODE B: $E_C \rightarrow t_1$; $t_1 \rightarrow t_3$; $t_3 \rightarrow t_2$; $t_3 \rightarrow t_4$; $t_4 \rightarrow E_A$

NODE C: $E_A \rightarrow t_5$; $t_5 \rightarrow t_6$; $t_6 \rightarrow t_8$; $t_6 \rightarrow t_1$; $t_1 \rightarrow E_B$

Apply the Obermarck algorithm and determine if a deadlock is detected, under the two following conventions:

(1) A node X transmits to a downstream node Y the arc $t_j \rightarrow t_i$ if $j > i$. (4 points)

(2) A node X transmits to a downstream node Y the arc $t_j \rightarrow t_i$ if $j < i$. (4 points)

(3) Consider schedules comprising abort requests and explain why long duration write locks (i.e., strict 2PL locks for write operations) are required. (3 points)

Solution.

Exercise solutions, point 1

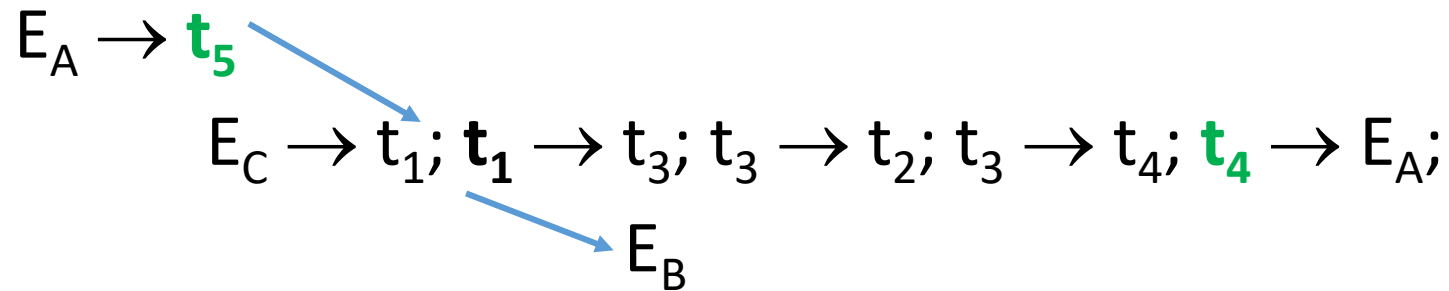
- Node A: $E_B \rightarrow t_4$; $t_4 \rightarrow t_7$; $t_7 \rightarrow t_5$; $t_5 \rightarrow E_C$; -- does not transmit
- Node B: $E_C \rightarrow t_1$; $t_1 \rightarrow t_3$; $t_3 \rightarrow t_2$; $t_3 \rightarrow t_4$; $t_4 \rightarrow E_A$; -- does not transmit
- Node C: $E_A \rightarrow t_5$; $t_5 \rightarrow t_6$; $t_6 \rightarrow t_8$; $t_6 \rightarrow t_1$; $t_1 \rightarrow E_B$; -- can transmit

Exercise solutions, point 1

- Node C:

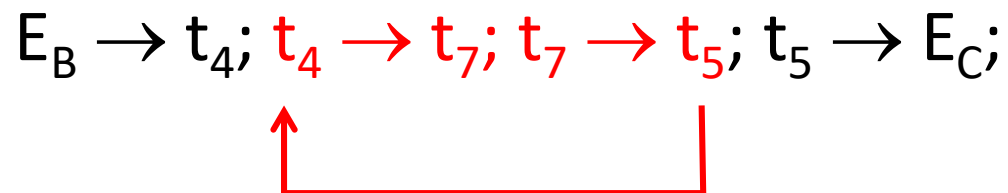
- $E_A \rightarrow t_5; t_5 \rightarrow t_6; t_6 \rightarrow t_8; t_6 \rightarrow t_1; t_1 \rightarrow E_B;$
- sends $E_A \rightarrow t_5; \rightarrow t_1 \rightarrow E_B;$ to Node B

- Node B



- Sends $E_A \rightarrow t_5 \rightarrow t_4 \rightarrow E_A$ to Node A

- Node A:

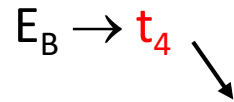


Exercise solutions, point 2

- Node A: $E_B \rightarrow t_4$; $t_4 \rightarrow t_7$; $t_7 \rightarrow t_5$; $t_5 \rightarrow E_C$; -- can transmit
- Node B: $E_C \rightarrow t_1$; $t_1 \rightarrow t_3$; $t_3 \rightarrow t_2$; $t_3 \rightarrow t_4$; $t_4 \rightarrow E_A$; -- can transmit
- Node C: $E_A \rightarrow t_5$; $t_5 \rightarrow t_6$; $t_6 \rightarrow t_8$; $t_6 \rightarrow t_1$; $t_1 \rightarrow E_B$; -- cannot transmit

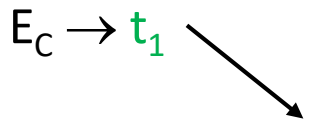
Exercise solutions, point 2

- Node C receives: $E_B \rightarrow t_4 \rightarrow t_5 \rightarrow E_C$



Node C: $E_A \rightarrow t_5; t_5 \rightarrow t_6; t_6 \rightarrow t_8; t_6 \rightarrow t_1; t_1 \rightarrow E_B$; -- cannot transmit

- Node A receives $E_C \rightarrow t_1 \rightarrow t_4 \rightarrow E_A$



Node A: $E_B \rightarrow t_4; t_4 \rightarrow t_7; t_7 \rightarrow t_5; t_5 \rightarrow E_C$; -- can transmit $E_C \rightarrow t_1 \rightarrow t_5 \rightarrow E_C$

Exercise solutions, point 2

- Node C receives $E_C \rightarrow t_1 \rightarrow t_5 \rightarrow E_C$

Node C: $E_A \rightarrow t_5; t_5 \rightarrow t_6; t_6 \rightarrow t_8; t_6 \rightarrow t_1; t_1 \rightarrow E_B;$



B. Ranking (12 points)

Dataset D is vertically distributed over three data sources (A , B , C , shown below), with values in $[0,1]$. Your task is to find the two items with the highest score according to function $f(t) = 4t.A + 2t.B + t.C$.

A	B	C
1: 1.00	2: 0.80	2: 0.60
7: 0.80	1: 0.65	7: 0.60
2: 0.70	3: 0.55	3: 0.50
3: 0.20	4: 0.50	1: 0.15
6: 0.15	5: 0.30	5: 0.10
4: 0.10	6: 0.30	4: 0.00
5: 0.10	7: 0.30	6: 0.00

1. Do this by applying FA. [4 points]
2. Do this by applying NRA. Show the buffer after each round. [5 points]
3. Could NRA ever make fewer sorted accesses than FA? Explain. [3 points]

You should indicate the **depth**, the number of **accesses**, the **results**, and succinctly show enough details so as to allow understanding each round of execution as well as the **stopping criterion**.

Solution.

1.
 - Sorted access phase: 4 rounds until $k = 2$ common items are found (here, 1, 2, 3 are in common).
 - Random access phase: r.a. to A : 4, r.a. to C : 4, r.a. to B : 7
 - Depth: 4. Sorted accesses: 12. Random accesses: 3.
 - Top-2 items: 1 and 2. The final table of retrieved items is as follows:

item	A	B	C	score
1	1.00	0.65	0.15	5.45
2	0.70	0.80	0.60	5.00
7	0.80	0.30	0.60	4.40
3	0.20	0.55	0.50	2.40
4	0.10	0.50	0.00	1.40

2.
 - Round 1. Buffer:

item	A	B	C	lb	ub
1	1.00	?	?	4.00	6.20
2	?	0.80	0.60	2.20	6.20
T	1.00	0.80	0.60	τ : 6.20	

- Round 2. Buffer:

item	A	B	C	lb	ub
1	1.00	0.65	?	5.30	5.90
7	0.80	?	0.60	3.80	5.10
2	?	0.80	0.60	2.20	5.40
T	0.80	0.65	0.60	τ : 5.10	

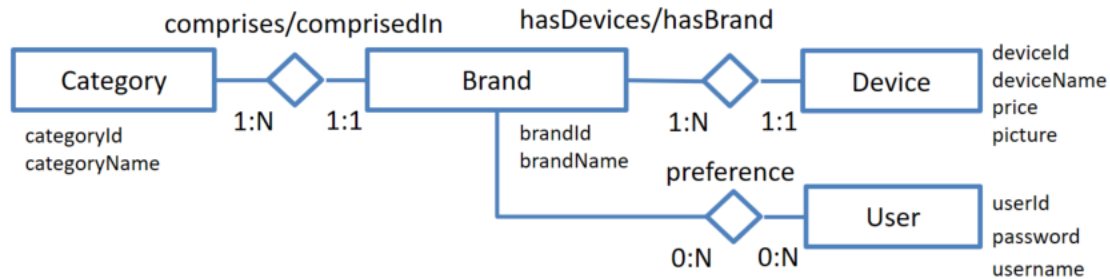
- Round 3. Buffer:

item	A	B	C	lb	ub
1	1.00	0.65	?	5.30	5.80
2	0.70	0.80	0.60	5.00	5.00
7	0.80	?	0.60	3.80	4.90
3	?	0.55	0.50	1.60	4.40
T	0.70	0.55	0.50	τ : 4.40	

- We can now stop because, in the buffer sorted by lb , the second-best lb (5.00) beats the highest ub value (4.90) after the second position as well as the threshold ($\tau = 4.40$).
 - Depth: 3. Sorted accesses: 9. Random accesses: 0.
 - Top-2 items: 1 and 2.
3. Yes, it happens here (9 vs 12 accesses). NRA exploits the scoring function, while the sorted access phase of FA does not, so NRA might meet the stopping criterion before FA does, thereby achieving a smaller depth. The counterpart is that, for this to happen, NRA has not seen all the scores of the top- k items (or else k items would have been completely seen by FA, too, which would then have stopped at the same depth), so the final scores of some items are uncertain ($lb \neq ub$, like for item 1 here).

C. JPA (9 points)

An e-commerce application manages a catalog of electronic devices belonging to different categories (desktops, laptops, tablets, phones) and brands (HP, Dell, Apple, Samsung..). Every brand belongs exactly to one category. The user can explore the catalog, select a category to see its brands and select one brand to see its devices. From a device, the user can get back to the brand. The user can also define preferred brands. The categories are in the order of tens, the brands in the order of hundreds, the devices in the order of tens of thousands and the users in the order of millions. Consider the ER model below.



With regard to the JPA rendering of the above schema, fill in the table below to specify, for each relationship, whether it is required, its annotation (e.g., @OneToMany), the mapped-by attribute, the owner side, the fetch policy and the cascading policies you consider appropriate to support the access required by the web application. Then, show the code of the JPA entity that maps the Brand entity in the conceptual model, taking into account the above-mentioned access paths of the application and data cardinality. Comment on your design choices.

Solution.

Relation Comprises: Category → Brand	
Required (with motivation)	YES, to select a category and see its brands
Annotation	@OneToMany
Owner	Brand
FetchType	Can be eager
CascadeType	REMOVE
Relation ComprisedIn: Brand → Category	
Required (with motivation)	not necessary, can be mapped for consistency
Annotation	@ManyToOne
FetchType	Eager (default)
CascadeType	not necessary
Relation hasDevices: Brand → Device	
Required (with motivation)	YES, to select one brand and see its devices.
Annotation	@OneToMany
Owner	Device
FetchType	Can be lazy
CascadeType	REMOVE
Relation hasBrand: Device → Brand	
Required (with motivation)	YES, is used to get back from the device to the brand
Annotation	@ManyToOne
FetchType	Eager (default)
CascadeType	not necessary

Relation Preference: User $\rightarrow$ Brand	
Required (with motivation)	YES, to see one's preferred brands
Annotation	@ManyToMany
Owner	Either User or Brand
FetchType	Can be Eager
CascadeType	None
Relation Preference: Brand $\rightarrow$ User	
Required (with motivation)	not strictly necessary, can be mapped for symmetry
Annotation	@ManyToMany
FetchType	Lazy
CascadeType	not necessary

```

@Entity
public class Brand{
    @Id @GeneratedValue(strategy=GenerationType.AUTO)
    private int brandId;
    private String brandName;

    @ManyToOne
    @JoinColumn(name="category")
    private Category category; // owner of the relation

    @OneToMany(mappedBy="brand", fetch = FetchType.LAZY, cascade=CascadeType.REMOVE)
    private List<device> devices;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable( name="usr_brand", joinColumns={
        @JoinColumn(name="brandid")}, inverseJoinColumns={@JoinColumn(name="userid")})
    private List<User> interestedUsers;
    //getters and setters...
}

```